WEEK 4 READ IN ORDER — THIS WEEK THE MODEL LOOKS BACK

From attention to transformers

What RNNs still cannot do, the one idea that fixed it, and the architecture that kept the idea and dropped the RNN.

TODAY · 12:30 – 3:30 · BREAK AT 2:15

One argument in three steps, then the family tree

12:40

Quiz 4

Honorlock · locks 12:55

12:55 · 01

Where reading in order stops

One message, three jobs · RNN, LSTM, GRU · the one-memory bottleneck

1:20 · 02

Attention

Let the model look back · what it fixed, what it did not

1:45 · 03

The transformer

Attention without the RNN, one idea at a time · break 2:15

2:25 · 04

Encoders and decoders

Three shapes · what each is for · today's small models

2:55 · 05

A language model on your laptop

Anatomy · what it costs to run · small vs large · lab: one model of each shape

3:15

Wrap-up

Workflow case (7 min) · Assignment 5 · Quiz 5

One message, three jobs

> “The replacement arrived, but it was damaged. I do not want another replacement. Please refund the charge.”

WEEK 3 COUNTED WORDS

replacement ×2 · refund ×1 · damaged ×1. A count leans the wrong way: the word the customer does *not* want is the most frequent one.

WEEK 4 READ IN ORDER

The reader keeps the order and the *not* — and we watched its memory fade by the end of a long ticket.

JOB 1 · ROUTE

Which team gets it?

One label from the whole message.

JOB 2 · EXTRACT

What does the customer want?

A refund, not a replacement — the *not* decides it.

JOB 3 · DRAFT

Write the reply

It must not offer another replacement.

◆ Today’s two tools: an **encoder** to read this message and route it, a **decoder** to write the reply. Block 4 says which is which, and why.

Three readers, one idea: read in order, keep a memory

READER	WHAT IT ADDS	LEARNED NUMBERS	TICKET ROUTER
`nn.RNN`	one running memory, rewritten completely at every word — fades within about ten words	8,320	0.78
`nn.LSTM`	three **gates** (forget, input, output) and a protected second memory, so something can be **kept**	33,280	**0.97**
`nn.GRU`	the same protection with two gates and one memory — simpler arithmetic, about three quarters of the cost	24,960	matched the LSTM in Assignment 4

WHAT THE GATES FIXED

The memory fades far more slowly

A gate can choose what to keep, so an early request can survive to the end of a long ticket. Slower fading, not none.

WHAT THEY DID NOT CHANGE

One word at a time, one memory

Every reader still squeezes everything it has read into a single fixed-size list of numbers.

Last week’s numbers: 64 numbers in, a 64-number memory; router accuracy on the Northwind test tickets.

Three limits the gates did not remove

LIMIT 1

Everything must fit in one memory

However long the input, the model's whole understanding of it is squeezed into one fixed-size list of numbers. A paragraph and a sentence get the same 64.

LIMIT 2

It cannot look back

Once a word has gone by, the only trace of it is whatever survived in the memory. The model can never re-read the input to check a detail.

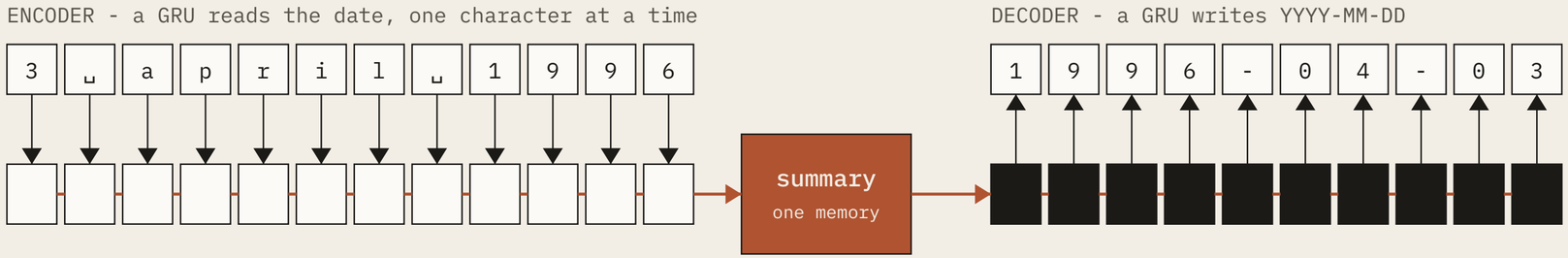
LIMIT 3

It reads strictly in order

Word 50 cannot be processed until words 1 to 49 are done. A GPU with thousands of cores mostly waits.

◆ One label is a small thing to remember, so the router survived. The trouble starts when the **answer is a sequence too**: translate, summarise, rewrite.

Two readers back to back: an encoder reads, a decoder writes



THE TASK · DATES WRITTEN A DOZEN WAYS → YYYY-MM-DD

```
received on 12 december 1994      -> 1994-12-12
received on 3 april 1996          -> 1996-04-03
on october the 25th , 2000        -> 2000-10-25
jan 20 2012                       -> 2012-01-20
30 jan 2022                       -> 2022-01-30
```

- The task: 6,000 dates written a dozen ways → `YYYY-MM-DD`. Everything the decoder knows about the input passes through **one summary**.

One summary is a bottleneck

TEST DATES REWRITTEN EXACTLY RIGHT

3.6%

GRU encoder–decoder, no attention · memory of 16 · 12,317 learned numbers · 1,000 test dates

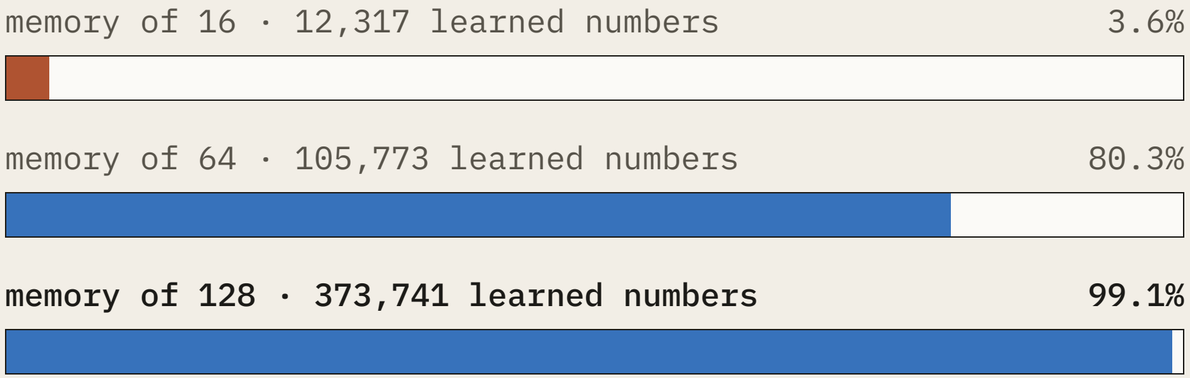
WHAT IT WROTE

```
may. 6, 1995                    should be 1995-05-06   wrote 1995-05-05
on july the 17th , 2012         should be 2012-07-17   wrote 2017-07-17
wednesday the 20th of february 2030 should be 2030-02-20   wrote 2003-02-23
on november the 17th , 2017     should be 2017-11-17   wrote 2017-11-11
```

READ THE ERRORS

The format is right and the answer is often close. What is missing is the *detail*: which day, which month. It all had to fit through one small memory.

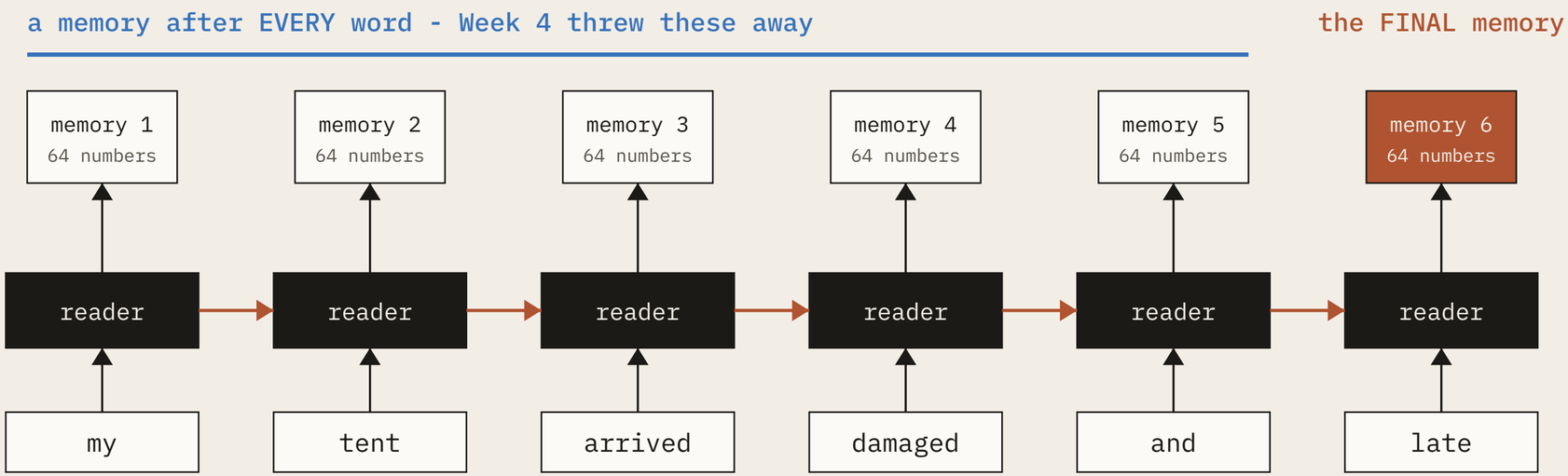
Could we just buy a bigger memory?



- → It works — at about **30 times** the learned numbers.
- → It does not last. A date is 30 characters. A ticket is 300; a contract is 30,000.
- → The **design** is the problem, not the size: a fixed-size summary of an input of any length.

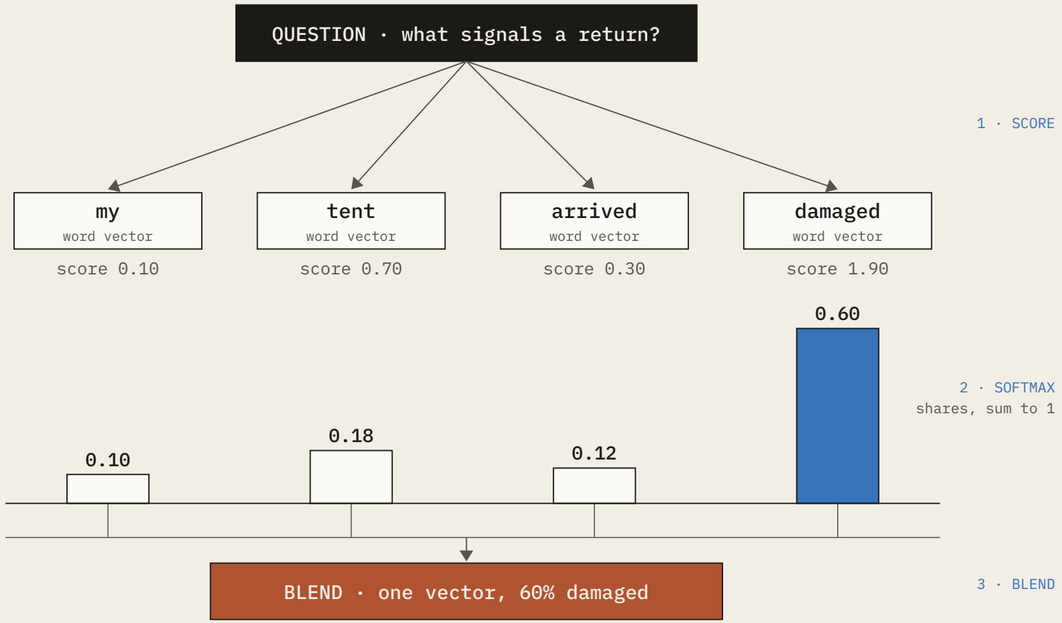
◆ A bigger summary is not the answer. Look instead at what the encoder **already produced** — and threw away.

The reader already made a memory after every word. We kept one.



- The router used the last 64 numbers; the date model the last 16. Keep the **whole row** and let the decoder look at it.

Attention: weigh every memory by how much it matters to the question



1 · SCORE

Ask every memory one question: *how relevant are you to what I am doing now?* One number each.

2 · SOFTMAX

Turn the numbers into shares of a budget: all positive, and they add up to 1.

3 · BLEND

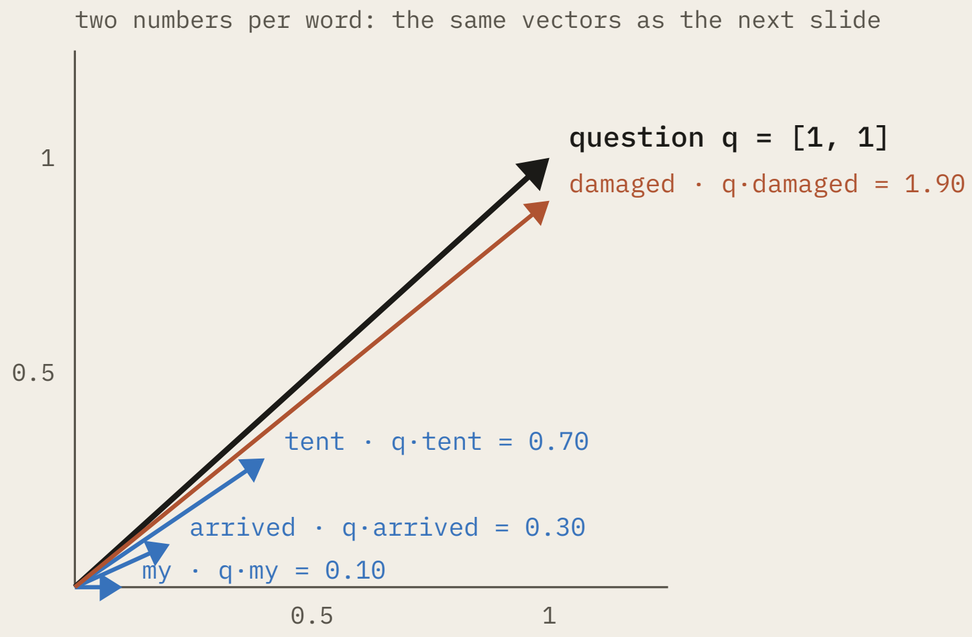
Average the memories using the shares. One vector comes out, made mostly of the relevant ones.

WHY IT BEATS ONE SUMMARY

Change the question and the same memories give different shares and a different summary. A fixed memory could never do that.

- A summary built **for this question**, and a new one for every question — instead of one summary that must serve them all.

Aside: the dot product measures agreement



WHAT THE NUMBER MEANS

a · b = |a| × |b| × cos(angle). Two things at once: **how long** the two vectors are, and **how aligned** they are.

WORD	q · WORD	CEILING |q| · |w|	COSINE	ANGLE TO q
my	0.10	0.14	0.707	45°
tent	0.70	0.71	0.990	8°
arrived	0.30	0.32	0.949	18°
damaged	1.90	1.90	0.999	3°

MAXIMUM, ZERO, NEGATIVE

Maximum |a| × |b| at 0°, aligned; longer vectors score higher, so no fixed ceiling across pairs. **Zero** at 90°: perpendicular, nothing in common. **Negative** past 90°, to −|a| × |b| at 180°. Unit vectors pin it to −1 … 1: Week 2's cosine.

- Length × length × cos(angle): biggest when aligned, zero at 90°, negative past it. That is attention's **relevance**.

The same three steps, by hand

WORD	ITS VECTOR	1 · SCORE = DOT WITH QUESTION [1, 1]	2 · SHARE = SOFTMAX(SCORE)
my	[0.1, 0]	0.1×1 + 0×1 = **0.10**	0.099
tent	[0.4, 0.3]	0.4×1 + 0.3×1 = **0.70**	0.181
arrived	[0.2, 0.1]	0.2×1 + 0.1×1 = **0.30**	0.121
damaged	[1, 0.9]	1×1 + 0.9×1 = **1.90**	**0.599**

share = $e^{\text{score}} \div (e^{0.10} + e^{0.70} + e^{0.30} + e^{1.90})$; the four add to 1. The code calls it *weight*.

3 · BLEND · WEIGHTED AVERAGE OF THE FOUR VECTORS

0.099×my + 0.181×tent + 0.121×arrived + 0.599×damaged = **[0.706, 0.606]** — mostly *damaged*, a little of everything else.

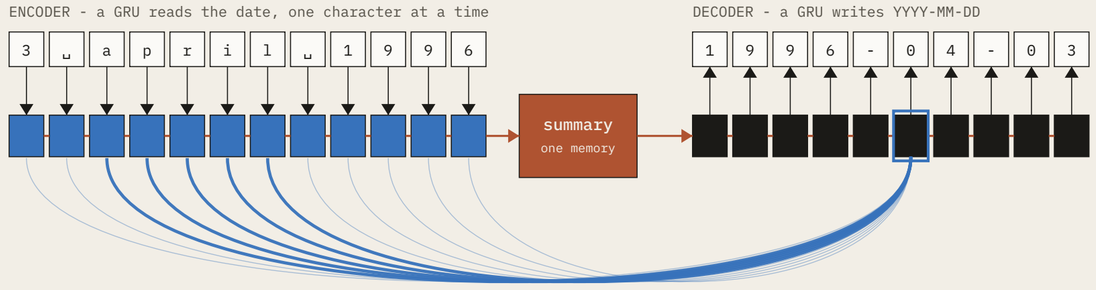
```
scores = word_vecs @ question
weights = np.exp(scores) / np.exp(scores).sum()
blend = weights @ word_vecs
```

THE SAME TABLE, PRINTED BY THE CODE

```
        my: score 0.10   weight 0.099   ###
      tent: score 0.70   weight 0.181   #######
   arrived: score 0.30   weight 0.121   ####
   damaged: score 1.90   weight 0.599   ########################
```

- Nothing here is learned: change the question and the same four words give different shares.

Let the decoder look back at every input memory



writing the month: look back at EVERY input memory, and weight the ones that matter now

THE QUESTION

The decoder's state right now: it is about to write the month, so its question is, in effect, *where is the month?*

THE MEMORIES

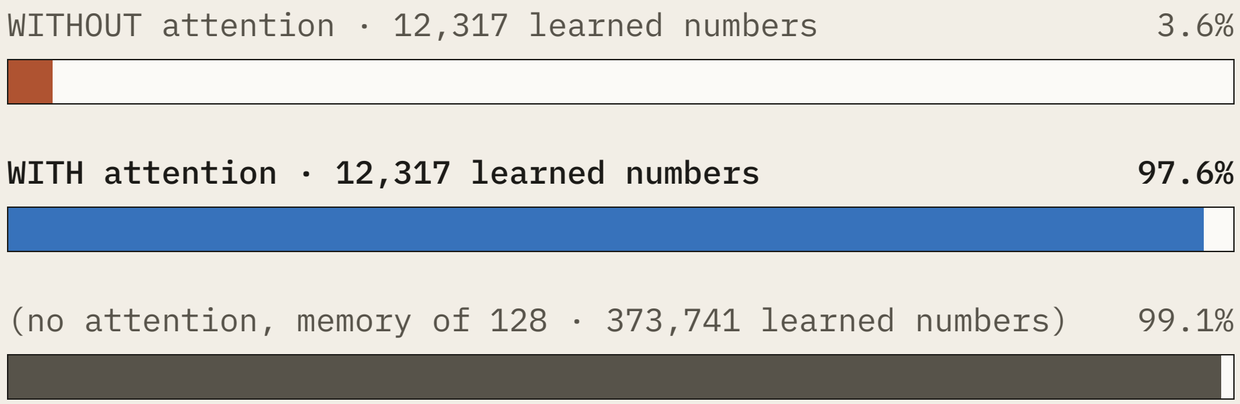
The encoder's blue row, one memory per input character — the row we were throwing away two slides ago.

THE BLEND

Handed to the decoder together with its own state; then it writes the character. A fresh question, and a fresh blend, at every step.

◆ Score every memory against the decoder's state, softmax, blend — the three steps from the last two slides, wired into the model. Nothing else changes.

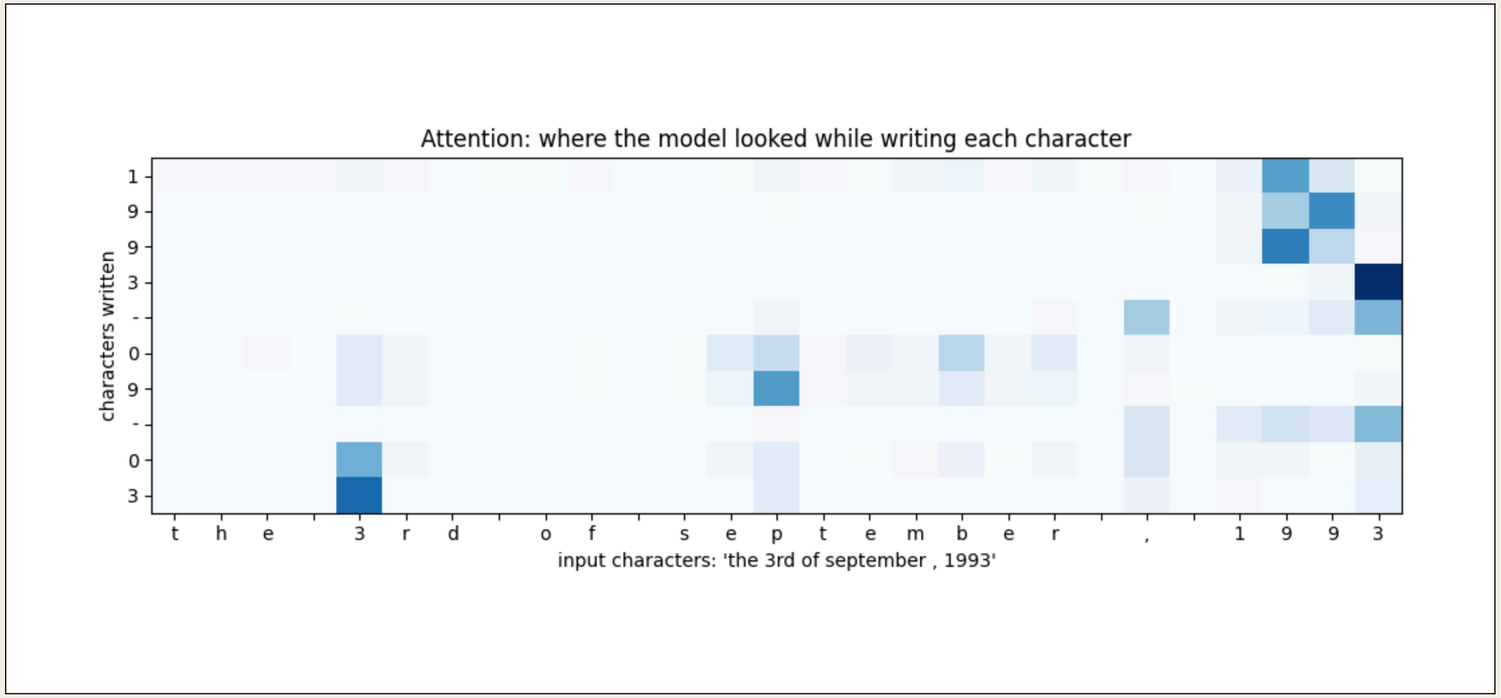
Same size, same data, same training: one new connection



- **Identical size.** The attention layers were in both models; one was allowed to use them.
- Matches the 128-memory model with about **a thirtieth** of the learned numbers.
- This is the paper that introduced attention: **Bahdanau, Cho & Bengio, 2014** — a bidirectional GRU encoder, a GRU decoder, attention between them. Attention was born inside RNNs.

◆ *Neural Machine Translation by Jointly Learning to Align and Translate*, Sept 2014 / ICLR 2015. Not the first attention-like idea (Graves, 2013), but the one the field built on; 2017 kept it and dropped the RNN.

Nobody told it where to look



`WRITING '1' (CHARACTER 1)`
looked hardest at input '9' (weight 0.54)

`WRITING '3' (CHARACTER 4)`
looked hardest at input '3' (weight 0.98)

`WRITING '9' (CHARACTER 7)`
looked hardest at input 'p' (weight 0.56)

`WRITING '3' (CHARACTER 10)`
looked hardest at input '3' (weight 0.76)

- The output reorders the input — year first, day last — and attention looks wherever the answer is. A weight of 0.7 is **70% of the blend**, not 70% confidence.

Two limits fixed. The RNNs still read in order.

LIMIT 1 · ONE MEMORY

Fixed. The decoder blends all the encoder's memories, afresh for every character it writes.

LIMIT 2 · CANNOT LOOK BACK

Fixed. Looking back is what attention is.

LIMIT 3 · STRICTLY IN ORDER

Not fixed. Attention sits between two RNNs. The encoder still reads one character at a time; the decoder still writes one at a time.

MEASURED · ONE LAYER, 8 TEXTS · MS, LOWER IS BETTER · LAPTOP GPU = APPLE (MPS)

LENGTH	GRU · CPU	GRU · GPU	ATTENTION · GPU
64 words	0.6 ms	5.6 ms	0.2 ms
256 words	2.3 ms	19.2 ms	0.3 ms
1,024 words	9.0 ms	66.9 ms	3.1 ms
4,096 words	36.3 ms	263.6 ms	45.8 ms

- → The GRU **cannot use a GPU**: 264 ms there vs 36 ms on the CPU. Step 50 waits for step 49; thousands of cores sit idle.
- → Attention has **no steps to wait for**: every word is compared with every other at once, so 4,096 words take 46 ms on the same GPU.

◆ If attention can look anywhere, why keep the RNN **at all**? 2017's answer is a paper title — *Attention Is All You Need* — and Block 3 builds that machine.

One vector for "charge" — but three meanings

WEEK 4 · THE LOOKUP TABLE HAS *ONE* ROW FOR "CHARGE" — THE SAME STARTING VECTOR IN ALL FOUR SENTENCES

"there is a second **charge** on my card for the same order"
① card charge · a payment

"please refund the extra **charge** you took from my account"
② extra charge · a payment

"the headlamp battery will not hold a **charge** anymore"
③ battery charge · stored energy

"who is in **charge** of the returns desk this week"
④ in charge · responsibility

WHAT WE NEED · A DIFFERENT VECTOR FOR EACH USE · A TRAINED TRANSFORMER (BERT) ALREADY DOES THIS · 1.00 = IDENTICAL

"CHARGE" IN …	① CARD	② EXTRA	③ BATTERY	④ IN CHARGE
① card charge	1.00	**0.80**	0.61	0.42
② extra charge	**0.80**	1.00	0.58	0.43
③ battery charge	0.61	0.58	1.00	0.36
④ in charge	0.42	0.43	0.36	1.00

WHAT TO SEE

Same starting vector, four different results. The two payment uses land together (0.8); the battery use is further off (0.61); "in charge" is furthest (0.42). Nobody labelled the meanings — the surrounding words moved the vectors.

◆ The fix, in one line: start from the lookup vector, then **let the words update each other** until each vector means what the word means *here*. The rest of this block is how.

Every word asks a question; every word offers an answer

the | damaged | blue | jacket | arrived | late

THE QUESTION · query

jacket asks

“Are there any adjectives just before me?”

A vector made from *jacket*’s own vector by a small learned layer. Every word asks its own question with the same layer.

THE ANSWER · key

every word replies

“damaged: yes. blue: yes. arrived: no.”

A second learned layer turns each word into an answer. Question and answers are compared with block 2’s dot product and softmax — *jacket*’s attention budget goes to *damaged* and *blue*.

WHAT TO ADD · value

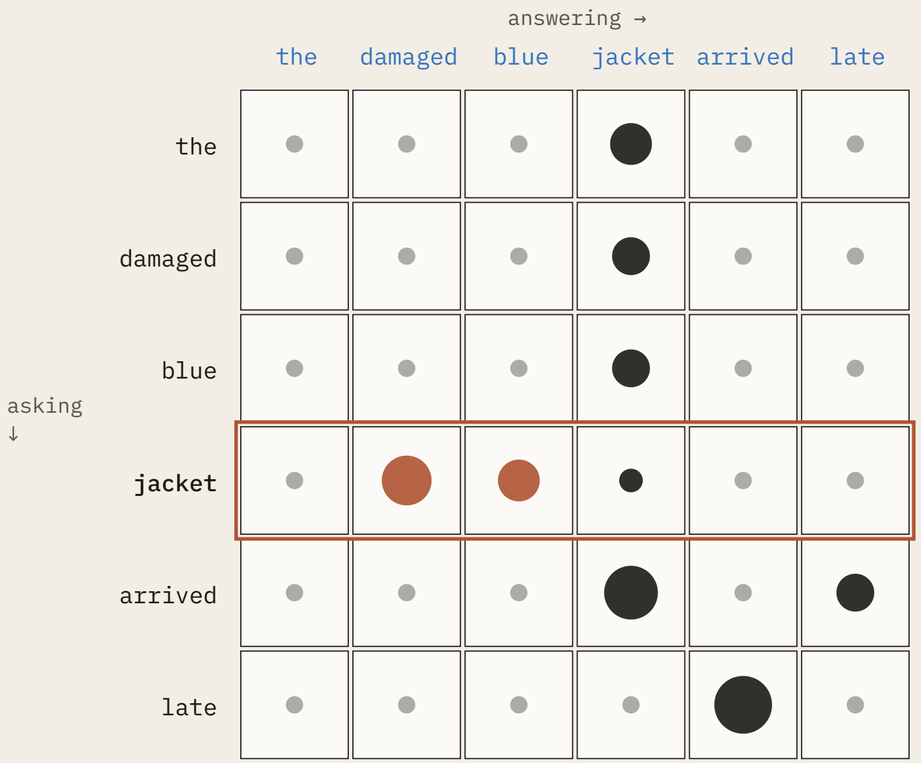
the matches contribute

“nudge jacket toward ‘a damaged thing’, ‘a blue thing’”

A third learned layer says what each word would contribute if picked. The weighted contributions are **added to** *jacket*’s vector — updated, not replaced.

◆ A story of what it *might* learn, to make the structure memorable. What a trained model actually learns is a research question — but the machinery is exactly this.

Every word asks at once: one table, one matrix multiplication



READ A ROW

jacket's row spends its budget on *damaged* and *blue*; *late*'s row looks back at *arrived*. Every row sums to 1. (Drawn to show the shape, not measured.)

THE UPDATE, IN ONE LINE

```
jacket' = jacket + 0.45·value(damaged) + 0.35·value(blue) +
…
```

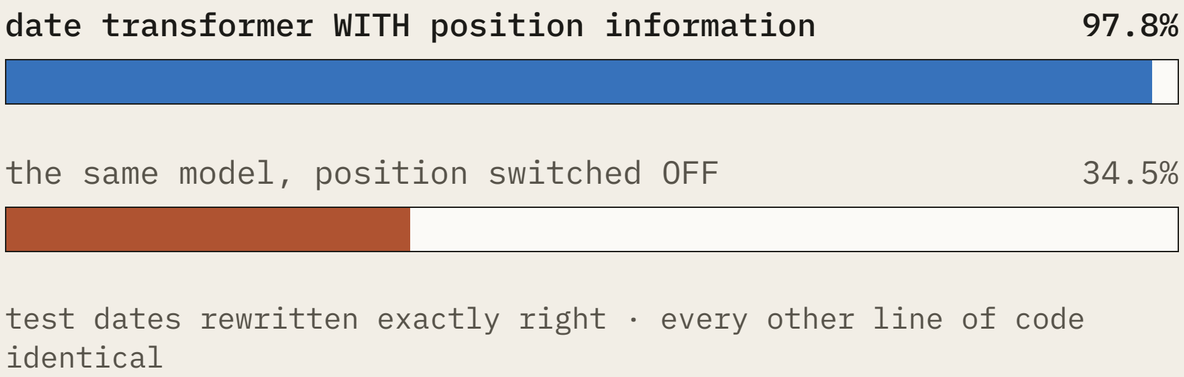
Updated, not replaced. Hold that plus sign for the block.

NO WORD WAITS

The whole table is one matrix multiplication. That single fact is what lets a transformer train across a whole GPU — and it is where the square-of-the-length cost comes from.

PROBLEM · ALL AT ONCE LOSES THE ORDER

Attention alone is a bag of words. Order must be added on purpose.



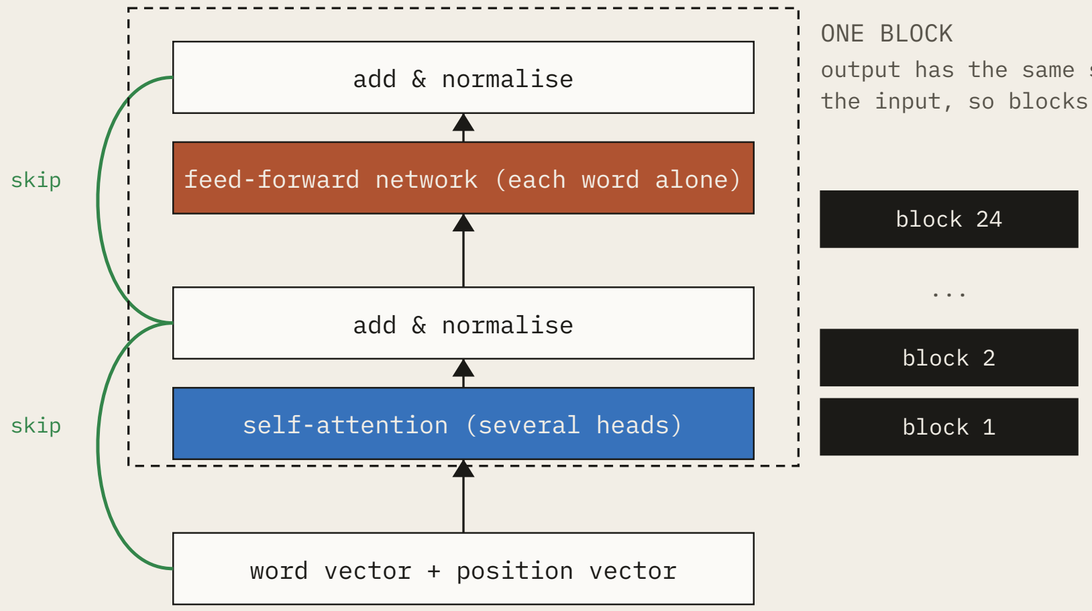
test dates rewritten exactly right · every other line of code identical

- Nothing in score–softmax–blend says **where** a word sits: **2026** and **2062** are the same four characters, shuffled.
- An RNN gets order for free. A transformer reads everything at once, so **position is added to the input**: word vector + position vector.
- How it is added has changed over time (sinusoidal → learned → RoPE); that detail is reference material in the notebook.

◆ Without position it sees a **bag of characters**: a 1, two 9s and a 3 are present — but not which comes first. Week 3's problem, again.

PROBLEM · ATTENTION ONLY MOVES INFORMATION

One block: words pull in context, then each word consults what it knows



SELF-ATTENTION · CONTEXT

The words exchange information — the only place one word learns anything from another. The plus sign from two slides ago: the block *adds* to each vector.

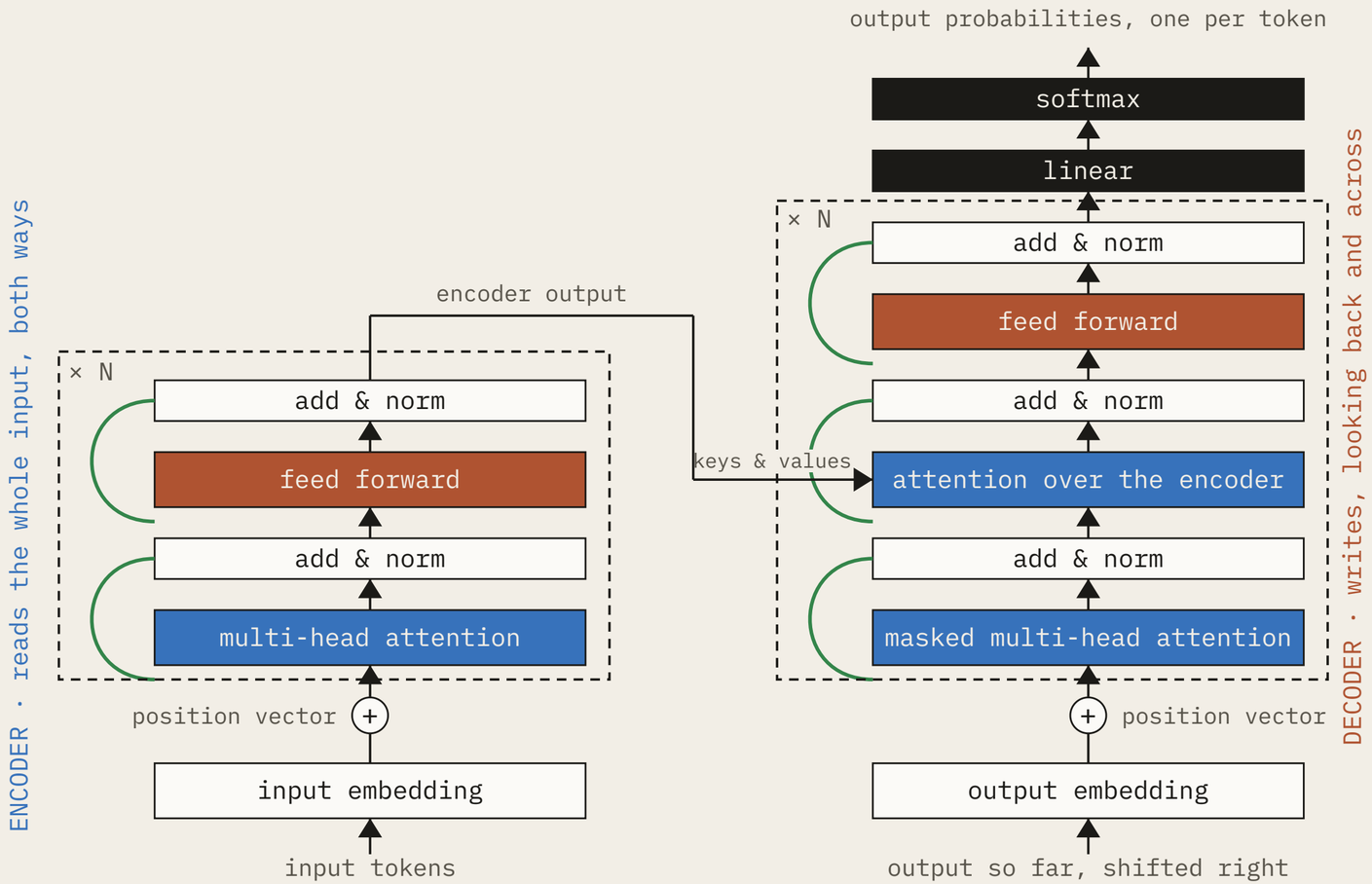
FEED-FORWARD · WHAT THE MODEL KNOWS

Week 3's two-layer network, applied to each word alone — **66.6%** of the block's learned numbers. Attention pulls in *context*; this is where *knowledge* lives: "the Ember stove runs on ___" is answered from here, not from any other word.

SKIP + NORMALISE

Keeps the signal alive through a deep stack — the same argument as the LSTM's protected memory.

The 2017 diagram, box by box



TWO TOWERS OF THE SAME BLOCK

Embedding + position at the bottom, then N copies of the block: **attention** pulls in context, **feed-forward** consults what the model knows, add & norm around each. 2017: N = 6. Our date model: N = 2. Qwen2.5-0.5B: 24.

THREE BOXES YOU HAVE NOT SEEN DRAWN

Masked attention: the writer may not look ahead — the notebook proves it (§3.2). A **second attention** that reads the encoder's output: Block 2's attention, kept. **Linear → softmax**: Week 4's output layer — one score per token, then probabilities.

WHERE BLOCK 4 GOES

Cut the diagram three ways: the left tower alone (**BERT**), the right tower without its middle attention (**GPT, Qwen**), or both (**T5, Whisper**).

The same machine as inputs and outputs

ENCODER · RUNS ONCE

Reads the whole input, hands over its meaning

IN

The input text as **n** tokens. Each id becomes a vector of width **d** from the lookup table, plus a position vector.

OUT

n vectors of width **d**, one per input token, each now carrying its context. This is the meaning of the input. It is computed once and does not change for the rest of the run.

DECODER · RUNS ONCE PER OUTPUT TOKEN

Writes one token, then runs again with it

IN

The **output so far**: a START token, then every token it has written, as **m** vectors of width **d** — plus the encoder's **n** vectors, read through its second attention.

OUT

One score per vocabulary token for the **next** position. Softmax, pick one, append it to the output so far — and run again, with one more token in. Stop at the STOP token.

n = input tokens · **m** = tokens written so far · **d** = numbers in every vector: 512 in the 2017 paper, 32 in our date model, 896 in Qwen2.5-0.5B · vocabulary: ~37,000 · the date characters · 151,936

◆ The encoder reads the whole input **once** and hands over its meaning. The decoder runs **again for every token** it writes, reading its own output back in.

Which shape for which job

Cut the 2017 diagram three ways — keep only the **encoder** (a reader: BERT, the embedding models), keep only the **decoder** (a writer: GPT, Qwen, Llama), keep **both** (a translator: T5, Whisper) — and you have the three kinds of model you can download. One question decides which you need.

IF THE JOB IS…	…THE TEXT	REACH FOR	BECAUSE
sort, tag, score, compare, **search**	already exists in full	**an encoder**	reads both directions; returns vectors, not prose; small and fast enough for millions of documents
write, chat, explain, follow instructions	does not exist yet	**a decoder**	trained to produce text one token at a time; one model, many tasks, through the prompt
convert one text (or audio) into another	input exists, output does not	**an encoder–decoder**	reads the whole input first, then writes — or, increasingly, a decoder with the input in its prompt

◆ Ask one question first: does the answer already **exist in the text**, or does it have to be **written**?

Today’s small language models, by shape

SHAPE	CURRENT SMALL MODELS	SIZE	WHERE YOU MEET THEM
`encoder-only`	BERT · DistilBERT · RoBERTa · **ModernBERT** · embedders: all-MiniLM, bge-small, e5, nomic-embed	20M – 400M	search and retrieval (Week 7), routers, spam and safety filters
`decoder-only`	**Qwen** 2.5 / 3 · **Llama 3.2** 1B, 3B · **Gemma 3** · Phi-4-mini · SmolLM 2 / 3	0.1B – 4B	local chat, on-device drafting, agents (Weeks 8 – 10)
`encoder–decoder`	T5 / Flan-T5 · BART · Marian, NLLB (translation) · **Whisper** (speech → text)	60M – 3B	translation, summarisation, transcription

THE FIVE ALREADY ON YOUR DISK · THE LAST COLUMN IS WHAT EACH CALLS ITSELF

model	shape	learned numbers	blocks	its own label
all-MiniLM-L6-v2	encoder-only	23M	6	BertModel
bert-base-uncased	encoder-only	109M	12	BertForMaskedLM
t5-small	encoder-decoder	61M	6 read + 6 write	T5ForConditionalGeneration
SmolLM2-135M-Instruct	decoder-only	135M	30	LlamaForCausalLM
Qwen2.5-0.5B-Instruct	decoder-only	494M	24	Qwen2ForCausalLM

A decoder becomes an assistant in two stages

THE SAME 0.5B DECODER, BEFORE AND AFTER INSTRUCTION TUNING

BASE MODEL, raw prompt:

```
' I bought a jacket last week and I want to return it. I have a receipt for it. I have a receipt for the jacket I bought last week. I have a receipt for the jacket I bought last week. I'
```

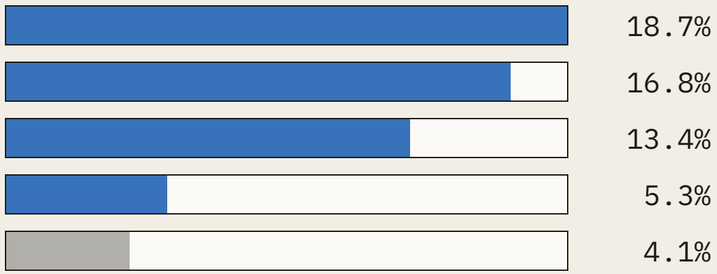
INSTRUCT MODEL, chat template:

```
'To return a jacket you bought last week, you can follow these steps:\n\n1. **Check the Return Policy**: Before you start the return process, make sure you understand the return policy of the store or the company that sold'
```

- **Pretraining**: predict the next word over a huge pile of text. The result continues text — it does not answer.
- **Instruction tuning**: examples of requests and good answers. This, not size, makes it an assistant.
- On the Hub the difference is one word in the name: `Qwen2.5-0.5B` vs `…-Instruct`.

Decoding: how the next token gets picked

AFTER “the package arrived damaged” · THE TOP 5 OF 151,936 PROBABILITIES



blue = kept by top-p 0.5 · top-k 5 keeps all five · greedy takes the first

GREEDY
Always the most likely token. Same prompt, same text, every time.

TEMPERATURE T
Divide the scores by T before softmax. T < 1 sharpens toward the top; T > 1 flattens toward a coin toss.

TOP-K
Keep only the k most likely tokens, then sample among them.

TOP-P
Keep the smallest set whose probabilities add up to p, then sample. Adapts to how sure the model is.

SETTING	2 SEEDS	ONE OF THE TWO REPLIES
greedy	identical	I am truly sorry for any inconvenience caused…
T=0.7, top-p=0.9	different	We are deeply sorry for the delay in your del…
T=1.5, no truncation	different	We are deeply sorry for the unpredictable del…
T=1.0, top-k=5	different	We are deeply sorry for the delay in your del…

◆ Greedy when a result must reproduce. Temperature, top-k or top-p for variety. Set them yourself.

Running a model: two bills, memory and time

BILL 1 · MEMORY TO HOLD THE MODEL = LEARNED NUMBERS × BYTES PER NUMBER

STORED AS	BYTES / NUMBER	0.5B MODEL	8B MODEL	70B MODEL
full precision	4	1.98 GB	**32 GB**	280 GB
half precision	2	0.99 GB	16 GB	140 GB
8-bit	1	0.49 GB	8.00 GB	70 GB
4-bit	½	0.25 GB	**4.00 GB**	35 GB

WHAT IT MEANS

An 8-billion model at full precision is **32 GB**: no laptop. Store each number in half a byte — **quantisation** — and it is **4 GB**: any laptop, at a small cost in quality.

BILL 2 · TIME TO WRITE, ONE TOKEN AT A TIME · QWEN2.5-0.5B WRITING 32 NEW TOKENS

TEXT SO FAR	RE-READS EVERYTHING	REMEMBERS THE PAST	TOKENS / S
16 tokens	0.93 s	**0.69 s** · 1.4× faster	47
256 tokens	1.47 s	**0.75 s** · 1.9× faster	42
1,024 tokens	4.02 s	**0.88 s** · 4.6× faster	36

WHAT IT MEANS

Every new token looks back at the whole text so far. Because a decoder's past never changes (the mask on the 2017 diagram), the library **remembers it instead of re-reading it**; the longer the conversation, the more that saves. The remembering is memory that grows with the conversation: 12 KB per token here, 0.403 GB at 32,768 tokens.

◆ Does it fit, and how fast will it write? Do this arithmetic **before you download anything** — Assignment 5, Task 1.

Small vs large is a measurement, not an opinion

24 SUPPORT TASKS, MECHANICALLY GRADED · THREE DECODERS · INSTRUCTOR'S LAPTOP

model	params (M)	fp32 weights (GB)	load (s)	tok/s	task-set (s)	accuracy
SmolLM2-135M-Instruct	134.5	0.54	0.8	80.9	8.9	0.250
Qwen2.5-0.5B-Instruct	494.0	1.98	0.9	67.0	4.3	0.708
Qwen2.5-1.5B-Instruct	1543.7	6.17	1.2	40.4	6.5	0.750

WHAT THE LADDER SHOWS

Accuracy climbs, speed falls

The 0.5B model is good at extraction and refusal and poor at routing, which needs two steps of reasoning. Next week starts from that failure.

THE REAL QUESTION

The smallest model that clears your bar

...and what it costs in latency, money and privacy exposure. You cannot answer it without a task set.

Design Northwind’s support workflow

Messages about billing, delivery, account access and returns. Wanted: faster routing and a drafted reply. Agents must approve refunds and policy exceptions.

1 · CHOOSE

Which shape for which job?

Routing, drafting: encoder, decoder, or a person? Name the baseline each has to beat — Week 3’s TF-IDF router is the honest one.

2 · EVALUATE

Which errors matter?

Two checks you would run. Per-queue accuracy on held-out tickets, rare queues included; a factual and policy check on every draft. One overall number hides the expensive mistakes.

3 · CONTROL

When must a person approve?

Refund amounts, policy exceptions, low-confidence routes. Decide the trigger before the model is built, not after the first complaint.

◆ 3 min discuss · 2 min compare with the next group · 2 min report a decision and why. Separate the jobs: an **encoder** to route, a **decoder** to draft, a **person** to approve.

EACH WEEK FIXES THE LAST ONE'S LIMIT

The climb so far

WEEK 2	**counts**	a document is which words it contains	limit: no meaning, no order
WEEK 4	**word vectors + reading in order**	meaning per word, a running memory across the text	limit: one memory; cannot look back; one word at a time
WEEK 5	**attention + transformers**	every word looks at every word, all at once; a vector per word *in context*	limit: cost grows with the square of the length
WEEK 7	**embeddings and retrieval**	encoders turn documents into vectors so a decoder is handed only what it needs	limit: next

This week

ASSIGNMENT 5 · DUE TUE SEP 29, 11:59 PM

Can we run this on our own hardware?

Inventory a small decoder before you run it · measure where the time and memory go · grade it on the 24-task support set · take control of its decoding · compare it with a frontier model and the invoice · write the recommendation. Everything it asks for is taught in notebook §4 – §6. Write-ups and the memo in your own words; AI Usage Report as always.

LAB · ONE MODEL OF EACH SHAPE · 40 MIN, NOTHING TO SUBMIT

Colab with one click, or your own laptop

Fill a blank with an encoder · embed five tickets · base vs instruct, temperature, tokens per second · translate and summarise with T5 · read what each model calls itself. Open in Colab from the course data repo, or set up locally with the getting-started guide (Miniconda, VS Code, your GPU). Start today if time allows; finish before the practice assignment.

READINGS

Three required

Jurafsky & Martin, ch. 8 *Transformers* · Alammar, *The Illustrated Transformer* · Vaswani et al., *Attention Is All You Need* (§3 only). Where attention came from: Bahdanau et al. (2014).

QUIZ 5 · WED SEP 30, 12:40

Covers today only

Six multiple choice, two fill-in, one written. Leave a few minutes for the written question.

APPENDIX

The code behind the slides

Read from the executed lecture notebook — what is shown is exactly what ran.

Why decoder-only won — and where the encoders went

REASON 1

One objective covers everything

“Predict the next word” needs no labels, so the training data is the whole internet — and any task can be phrased as *continue this*.

REASON 2

Its past never changes

So the past can be cached, and writing each new word is cheap. Block 5 measures what that buys.

REASON 3

No new layer per task

One model classifies, extracts, summarises and chats — the task arrives in the prompt. Weeks 6 to 10 depend on this.

◆ Encoders did not lose; they moved — to search, routing and filtering, where the answer is a **vector**, not a paragraph.

What changed, side by side

	RNN / LSTM / GRU (+ ATTENTION)	TRANSFORMER
how it reads	one word at a time, through a running memory	**all words at once**; every word looks at every word
word 1 → word 500	499 steps; the signal must survive each one	**one step**
word order	free — it reads in order	must be added to the input on purpose
on a GPU	mostly waits: each step needs the last	**works**: one big matrix multiplication
cost as text grows	in step with the length	with the **square** of the length — the bill we pay for the rows above
the task head	one score per label, from the final memory	**the same** — one score per label or per token, from the vectors
how you judge it	held-out tickets, accuracy per queue	**the same** — the architecture changed, the evaluation did not

- That last row returns twice: the running-cost slide later today, and Week 7, where it is the reason retrieval exists.

Is that only a story? Four lines compute the table — no loop

```
scores = Q @ K.T                        # (T, T) raw compatibility
d_k = d_model
scaled = scores / math.sqrt(d_k)        # the divisor that section 1.2 is about
attn = scaled.softmax(dim=-1)           # rows sum to 1
out = attn @ V                          # (T, d_model)

print("row sums of attn:", attn.sum(dim=-1).tolist())
print()
print(pd.DataFrame(attn.numpy().round(3), index=tokens, columns=tokens))
```

READ ONE ROW

Each row is one word's attention budget over all the words: positive, sums to 1. Untrained here, so the values are arbitrary — the shape is the point. Training is what turns these into the pattern on the previous slide.

ATTENTION WEIGHTS · ROW = THE WORD ASKING

	the	package	arrived	damaged	and	cold
the	0.134	0.173	0.180	0.097	0.238	0.179
package	0.372	0.032	0.099	0.264	0.070	0.162
arrived	0.058	0.209	0.069	0.282	0.155	0.227
damaged	0.034	0.103	0.076	0.254	0.337	0.197
and	0.048	0.198	0.279	0.102	0.227	0.146
cold	0.077	0.212	0.102	0.222	0.189	0.198

- Q, K and V are three `nn.Linear` layers — the question, the answer and what-to-add — applied to every word at once.

ONE QUESTION PER WORD IS ONLY ONE KIND OF RELEVANCE

One question per word is only one kind of relevance

THE PROBLEM

One table, one kind of relevance

jacket's question finds its adjectives. But who finds what *it* refers to? Which verb belongs to which subject? Which earlier sentence a later one contradicts? One attention table can hold one of these at a time.

THE SOLUTION · SEVERAL HEADS

Ask several questions at once

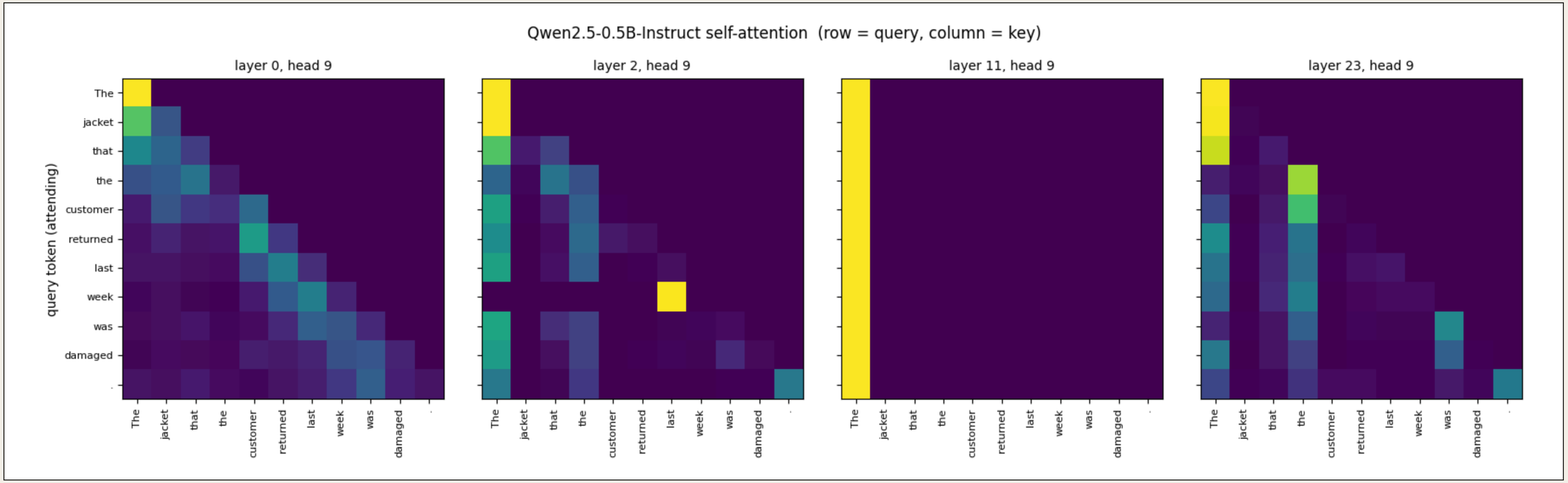
Run several attention tables side by side — *heads* — each with its own question, answer and what-to-add layers, and add all their contributions to the word. Real models use a dozen to a hundred heads in every block. Our by-hand version matches PyTorch's to 8.9×10^{-8}.

HOUSEKEEPING, NOT AN IDEA

Before the softmax the scores are divided by √d (the vector size), so long vectors do not force all the weight onto one word. It keeps training stable. That is the whole of it.

◆ Same recipe — score, softmax, blend — run a dozen to a hundred times side by side, each head asking a different question, all the answers added to the word.

Real attention, in a real model



- Qwen2.5-0.5B: 24 blocks. Early blocks look nearby; deep blocks jump. And nothing sits above the diagonal — it never looks **ahead**.

Attention in the date model: three lines

```
        for step in range(OUT_LEN):
            if self.use_attention:                                          # score every input position...
                scores = self.att_v(torch.tanh(self.att_q(state).unsqueeze(1) + self.att_k(enc_out))).squeeze(-1)
                w = F.softmax(scores.masked_fill(~real, -1e9), dim=1)   # ...turn scores into weights (sum to 1)
                context = (w.unsqueeze(-1) * enc_out).sum(1)            # ...and blend the input by those weights
            else:
                w = torch.zeros(x.shape)                                # switch OFF: the same summary every step
                context = summary
```

- `scores` → `softmax` → weighted sum of `enc_out`. Switch off: the same `summary` at every step.

The date transformer

```
class DateTransformer(nn.Module):
    """The date rewriter again - encoder blocks + decoder blocks, attention only, no RNN."""
    def __init__(self, d=32, heads=4, layers=2, use_position=True):
        super().__init__()
        self.use_position = use_position
        self.src_embed, self.tgt_embed = nn.Embedding(len(SRC), d, padding_idx=0), nn.Embedding(len(TGT), d)
        self.src_pos, self.tgt_pos = nn.Embedding(SRC_LEN, d), nn.Embedding(OUT_LEN, d)   # "where am I?" (§2)
        self.blocks = nn.Transformer(d_model=d, nhead=heads, num_encoder_layers=layers, num_decoder_layers=layers,
                                     dim_feedforward=4 * d, dropout=0.0, batch_first=True)
        self.out = nn.Linear(d, len(TGT))

    def forward(self, x, y_in):
        a, b = self.src_embed(x), self.tgt_embed(y_in)
        if self.use_position:
            a = a + self.src_pos(torch.arange(x.size(1)))
            b = b + self.tgt_pos(torch.arange(y_in.size(1)))
        no_peeking = nn.Transformer.generate_square_subsequent_mask(y_in.size(1))    # the causal mask (§3.2)
        h = self.blocks(a, b, tgt_mask=no_peeking, tgt_is_causal=True,
                        src_key_padding_mask=(x == 0), memory_key_padding_mask=(x == 0))
        return self.out(h)
```